LEARN · VERIFY — OFFLINE STUDY BOOKLET

Design for Deployment & Chaos Engineering

The system is never at rest. Ship into it without anyone noticing — then break it on purpose to find out what you actually built.

1
Deployment is a feature
The "during" state, the build funnel, and rebuilding versus repairing machines.

2
Make the data survive
Expand before, contract after; shims, translation pipelines, trickle-then-batch.

3
Roll it without being seen
Four time spans, grain size, canary batches, health checks, cleanup.

4
Prove it by breaking it
Chaos engineering: hypothesis, injection, targeting, blast radius, zombies.

Print double-sided, short-edge bind, A4 · 100% scale (no "fit to page").
Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

1. Read the plain-version box first. If it doesn't land, the rest won't either.
2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
5. Only then move on.

> **THE RULE THAT DOES MOST OF THE WORK**
>
> If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

The spacing schedule

Review at **day 1, day 3, day 7, day 16, day 35**. On a pass, jump to the next interval. On a fail, reset that item to 1 day. Use the grid at the back; one row per item.

What's in here

> **ONE WARNING BEFORE YOU START**
>
> Nothing here is a checklist to score yourself against. Both halves of this competency are judgment calls with a price attached: a human gate before production buys safety and costs speed; chaos in production buys knowledge and costs customer experience. If you finish able to recite the phases but not to say *what each one costs you*, the booklet has failed.

SECTION 1

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Given a real change to ship (“we need to split that table”, “the new version reads a different document shape”, “we deploy monthly and it always hurts”, “how do we know the failover actually works?”), you can:

1. **Name the phase** the work belongs in — expansion, rollout, or contraction — and say what breaks if you move it.
2. **Locate the compatibility risk**: which combination of old and new code against old and new data is the one that fails.
3. **Prescribe the mechanism** — shim, translation pipeline, trickle-then-batch, canary batch, health check, injection — and name the near-miss you rejected.
4. **State the price.** Every one of these costs latency, engineering time, customer experience, or speed.

The core model in one paragraph

Most design assumes the system has two states: **before** the release and **after** it. That assumes the whole system changes in one instantaneous quantum jump, and it doesn't. Updating takes time, and during that time both versions run at once — so there is a third state, **during**, and it is the only one your users actually get to see. Everything in the first three parts follows from taking “during” seriously: expand the schema before, roll code in batches, contract after. The fourth part is the other side of the same coin. Having designed a system you believe survives disruption, belief is not evidence — so you manufacture disruption, at a survivable size, while you are watching.

THE WHOLE THING FOR A SMART 15-YEAR-OLD

- Updating a live system is not a moment. It's a stretch of time where the old and new versions are both running.
- So the database and the code have to work for *both* versions at once — add things before, take things away only after.
- Update machines in small batches, watch the first batch, and give each one time to finish what it was doing.
- Then break your own system on purpose, in small controlled doses, because a thing that never breaks is a thing nobody knows how to fix.

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE
1 **Deployment is a feature**	The fallacy of planned downtime and the “during” state. Build pipelines as funnels. Configuration management, role mapping, convergence versus immutable infrastructure. Continuous deployment and the delay–risk cycle.	Everything downstream is a consequence of deciding that deployment is designed, not delegated. Settle that first.
2 **Make the data survive**	Expansion and contraction phases, shims, the four old/new read scenarios, translation pipelines, trickle-then-batch, web asset versioning and session affinity.	Data outlives code and cannot be rolled back cheaply. It is the hardest constraint on the “during” state, so it comes before the mechanics of moving code.

3 **Roll it out without being seen**	The four microscopic time spans, grain size, batches and canaries, health checks, immutable rollout choices, cleanup and feature toggles.	Only now is it safe to actually move code — the data can already tolerate a mixed fleet.
4 **Prove it by breaking it**	Chaos engineering: why whole-system, drift, the regulator paradox, hypothesis design, the three injections, targeting as a search problem, blast radius, disaster simulations.	Parts 1–3 produce a system you *believe* tolerates disruption. This part is how you find out, and it is empirical rather than formal on purpose.

> *“Releases should be like what Agent K says in Men in Black: ‘There's always an Arquillian Battle Cruiser, or Corillian Death Ray, or intergalactic plague, [or a major release to deploy], and the only way users get on with their happy lives is that they do not know about it!’”*

1

PART 1 OF 4

Deployment is a feature, not an event

The “during” state, the build funnel, and whether you repair machines or rebuild them.

TENTH-GRADER VERSION

- Shipping an update isn't a moment. It's a stretch of time, and during it two versions of your software are running side by side.
- Users don't care that you announced “planned downtime” in an internal email. To them, downtime is just downtime.
- So the update has to be something you *designed for*, not something you hand to someone else to figure out.
- And code sitting unreleased isn't safe — it's unfinished work piling up, getting riskier the longer it waits.

Start with the shop

- You own a shop and you want to renovate. Two options:
  - **Close for a week.** Simple to plan, and every customer who turns up finds a locked door. You told the staff, which is not the same as telling the customers.
  - **Renovate around them.** One aisle at a time, new shelves built before the old ones come out, nobody sent home. Harder — but the shop never stops earning.
- The second one only works if you **designed the renovation that way from the start**: new shelves that stand up on their own before the old ones are removed, and an order of work where the shop is always usable.
- That is the whole of this competency. “Planned downtime” is closing the shop and calling it a plan.

> *“I hate the phrase ‘planned downtime.’ Nobody ever clues the users in on the plan. To the users, downtime is downtime.”*

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Machine	One configurable operating-system instance — whatever the unit happens to be.	A physical host on real metal; a virtual machine; a container; a unikernel.	vs **service**: a service is a callable interface for others to use, and it is *always* made of redundant copies running on multiple machines. One machine is never a service.
Build pipeline	The automated path from a commit to a production deployment.	Compile → unit test → package → deploy trial → in-situ test → deploy real. Jenkins, GoCD, Spinnaker, AWS CodePipeline, or shell scripts and post-commit hooks.	vs **continuous integration**: CI stops after publishing a test report and an archive. The pipeline carries on into trial environments, migrations, integration tests and production. It's better understood as a **funnel** — every stage exists to find a reason to reject the build.

Configuration management	Declaring the *desired end state* of a machine in version-controlled text, and letting a tool work out the actions.	Chef and its peers. The description says what the machine should look like; the tool figures out the copying, templating, user creation and ordering.	vs a **shell script**, which specifies the actions themselves. The tool also owns **dependency ordering**: directories before files, users before file ownership.
Role mapping	Deciding which machine performs which role.	**Manual:** the operator says “server 1 is web, server 2 is app.” **Automatic:** the operator says “service X, Y replicas, across these locations” and the platform places them.	Automatic mapping goes hand in hand with platform-as-a-service: the platform matches requested capacity against constraints, finds hosts with enough CPU, RAM and disk, avoids co-locating instances, and must then configure load balancing and routing because the addresses aren't known in advance.
Convergence	Examine the machine's current state, work out a plan to reach the desired state, and apply it.	A long-lived VM that has survived many deployments gets a new recipe run against it.	vs **immutable infrastructure**: convergence starts from an *unknown* state. Critics point out it can be unreachable — some resources end up in states the tool can't repair — and that whatever isn't in your recipes (kernel parameters, TCP timeouts) is left untouched and may be nothing like you expect.
Immutable infrastructure	Never repair a machine. Build a new image, register it, delete the old machine.	The unit of packaging is a VM or container image, built entirely by the pipeline. Extra configuration is injected at startup — an AMI instance interrogates its environment for the “user data” supplied at launch.	You always start from a *known* state: the master OS image. So it should succeed every time, and debugging recipes is tractable because there is one initial state rather than the stucco-like accumulation of a long-lived machine. **Immutable is for cattle; convergence is for pets.**
Continuous deployment	Run the full pipeline on every commit, to shorten the gap between commit and production.	Some teams trigger the production deploy automatically; others insert a pause stage requiring a human to affirm the build.	vs **continuous delivery / integration**: the distinguishing act is reaching production. The motivation is that **undeployed code is inventory** — until it is in production you cannot know whether it has bugs, breaks scaling, or implements a feature nobody wants.

> *“Between the time a developer commits code to the repository and the time it runs in production, code is a pure liability. Undeployed code is unfinished inventory.”*

The vicious cycle worth memorising

- The longer the delay between deployments, the **more changes accumulate** in each one.
- A bigger deployment carrying more change is **riskier**.
- When that risk materialises, the natural reaction is to **add review steps** to prevent a recurrence.
- Review steps **lengthen the delay** — which increases risk further. The cure feeds the disease.
- There is exactly one way out, and it is counter-intuitive enough that it has to be internalised as a motto: **“If it hurts, do it more often.”** Taken to the limit, that means do everything continuously — for deployments, run the full pipeline on every commit.

WHY THIS IS THE HARDEST IDEA IN THE PART

Every instinct says that a risky operation deserves more scrutiny and less frequency. For deployments the opposite holds, because **frequency is what shrinks the batch**, and batch size is what actually carries the risk. The review step feels like it reduces risk while measurably increasing the thing that causes it.

Trade-offs — nothing here is free

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Automatic production deploy at the end of the pipeline	Shortest possible commit-to-production delay; smallest batches; the liability of undeployed code is minimised.	No human judgment between a green build and live traffic.	The cost of moving slowly exceeds the cost of a deployment error.
A human “pause” gate before production	Someone with context affirms the build — worded honestly, “yes, you may fire me if this fails.”	Longer delay, bigger batches, and you must guarantee an authorised button-pusher is available whenever a change is needed — **including a 2 a.m. emergency**.	Safety-critical or highly regulated work, where an error costs far more than moving slowly relative to competitors.
Convergence	Works on physical hosts and long-lived VMs; no image-building infrastructure needed.	Starts from an unknown state, so it can be unreachable; anything outside your recipes drifts silently.	Pets — physical deployments, long-lived VMs, manual role mapping.
Immutable infrastructure	Always starts from a known state, so it should succeed every time and debugging is tractable.	Requires image build and registration in the pipeline; all extra configuration must be injectable at startup.	Cattle — IaaS, PaaS, automatic role mapping.
Shopping for the best pipeline tool	Nothing, mostly.	Time. This is the analysis trap.	Never. **Pick one that suffices and get good with it.**

CONCEPT BOUNDARY — WHAT “DESIGN FOR DEPLOYMENT” *IS NOT*

It is not writing better release notes, or handing operations a cleaner binary, or scheduling the outage more considerately.
It is writing the application so that it functions correctly *while half the fleet is running the other version* — which is a design constraint on your schema, your assets and your startup behaviour, not an operational procedure.

Anchor sketch — the funnel and the two ways to reach a machine

Legend: boxed = a named node | **reversed = the node that matters most** | **Q = cover the page and answer this**

```
  commit
    |
    v
 BUILD PIPELINE -- a funnel: every stage looks for a reason to REJECT
    |
    +--> compile / unit test / analyse      (this much is CI)
    +--> package
    +--> deploy trial -> in-situ test       (CI stops before here)
    +--> deploy real
    |
    v
 CONFIG MANAGEMENT -- declare the END STATE, not the actions
    |
    +-- how does the machine get there? ------+
    |                                         |
 CONVERGENCE                            IMMUTABLE
 repair from an unknown state           rebuild from a known image
 "pets"                                 "cattle"
    |                                         |
    +-- who decides which machine does what? -+
    |                                         |
 manual mapping                          automatic mapping
 "server 2 is app"                       "service X, Y replicas"
```

Q Where exactly does CI end and the build pipeline continue — and why is “funnel” the better word?
Q Name the two failure modes convergence has that the reversed node does not.
Q Which pairing of the bottom two rows do these naturally align into, and why?

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“We'll take the site down Saturday night to release.”	“That's planned downtime, which is only planned from the inside. We should design for the ‘during’ state so the release is undetectable.”
“Our build server also deploys.”	“We have a build pipeline, not just CI — it continues past the test report into trial deploys, migrations, integration tests and production.”
“The script installs everything.”	“We use convergence: the tool inspects current state and plans a path to the declared end state. The risk is that it starts from an unknown state and leaves anything outside the recipes untouched.”
“We just build a new box instead of fixing the old one.”	“Immutable infrastructure. The unit of packaging is the image, extra config is injected at launch, and the outdated machine is deleted rather than repaired.”
“We don't say which server runs what.”	“Automatic role mapping — we declare replica counts and constraints, and the platform places instances and configures routing.”
“We release monthly because releases are risky.”	“That's the delay–risk cycle: longer gaps mean bigger batches, which raise risk, which invites review steps that lengthen the gap. If it hurts, do it more often.”
“It's written but not shipped yet.”	“Then it's unfinished inventory and a pure liability — unknown bugs, unknown scaling behaviour, unknown value until it's in production.”

Retrieval — cover the page first

PART 1 · QUESTION 1

A team deploys once a month. Each release takes a four-hour maintenance window, and roughly one in three causes an incident. After the last bad one, the team lead proposes adding a mandatory architecture review and a second sign-off before any release, and moving to a six-week cycle to fit them in.

(a) Name the cycle the team is about to accelerate, and state the mechanism in one sentence. **(b)** Explain why the proposal will measurably increase the thing it is meant to reduce. **(c)** Give the counter-proposal, and say what would have to become true about the application itself for it to be safe.

PART 1 · QUESTION 2

Two services in the same company both deploy cleanly in staging. In production, service A's deploys succeed almost always; service B's fail perhaps one time in five, each time for a different and unreproducible reason. A runs in containers rebuilt from a base image every deploy. B runs on VMs that have been alive for three years, updated in place.

(a) Name both approaches and the one-line difference between them. **(b)** Give the two distinct reasons B's failures are unreproducible — one about reachable state, one about what the recipes never mention. **(c)** Say what B gets in exchange for that pain, and which mapping style each approach naturally pairs with.

2×2 — how any fleet gets from “code exists” to “code is running”

COLUMNS: WHO DECIDES WHICH MACHINE RUNS WHICH ROLE? →

	Manual mapping an operator names the role per host	**Automatic mapping** “service X, Y replicas” and the platform places it
CONVERGENCE repair toward the desired state	**Pets** Physical hosts and long-lived VMs. The tool inspects the machine and plans a path to the declared state. *Move:* accept drift as real. Test recipes against messy, long-lived states, not fresh ones — and remember that anything outside the recipe is untouched.	**The awkward middle** A platform places workloads, but the machines underneath still accumulate state across deploys. *Move:* shorten machine lifetime until it is effectively immutable, or move packaging up to the image. Don't leave it here.
IMMUTABLE rebuild from a known image	**Baked images, hand-placed** AMIs or container images built by the pipeline, but an operator still decides placement. *Move:* inject every environment-specific value at startup, never at build time, or you will bake one image per environment.	**Cattle** IaaS and PaaS. The image is the unit of packaging; the platform matches capacity to constraints, avoids co-locating instances, and configures routing. *Move:* this is where zero-downtime rollout is cheapest, because replacing an instance *is* the deployment. Spend the effort here rather than perfecting convergence.

Read it as: the two diagonal corners are where these approaches naturally align — immutable closely with IaaS/PaaS and automatic mapping, convergence more commonly with physical hosts, long-lived VMs and manual mapping. The off-diagonal cells are transitional; if you find yourself in one, it is usually a sign of a migration that stalled halfway.

CARRY-OUT RULE FROM THIS PART

Treat deployment as a feature with a design, not a procedure with a runbook. The test is simple: if your application cannot function correctly while half the fleet runs the previous version, you have not designed for deployment — you have merely automated a downtime window.

2

PART 2 OF 4

Make the data survive the “during”

Expand before, contract after. Shims, translation pipelines, trickle-then-batch, and versioned assets.

TENTH-GRADER VERSION

- While you're updating, some machines run the old code and some run the new. The database has to keep both of them happy.
- So you only **add** things before the update, and only **remove** things after it's completely finished.
- The dangerous combination is old code meeting new data — it has never seen that shape and doesn't know what to do.
- “We don't have a schema” doesn't help. The database might not care about structure, but your code absolutely does.

Start with the bridge

- You need to widen a bridge that carries traffic all day. You cannot close it, and you cannot rebuild it in one night.
- So the order of work is forced: **build the new lanes alongside first** while the old lanes still carry everything. Then move the traffic across. Only when nothing is using the old lanes do you demolish them.
- Demolishing early is the one thing guaranteed to hurt someone, and it is exactly the mistake of dropping a column while old instances are still running.
- The temporary connectors between old and new lanes — the bits you build only so both can work at once, and remove afterwards — are **shims**.

The two phases, and what belongs in each

EXPANSION — SAFE *BEFORE* ROLLOUT	CONTRACTION — ONLY *AFTER* ROLLOUT
Add a table · add views · add a **nullable** column · add aliases or synonyms · add new stored procedures · **add triggers — but only if they are nonconditional and cannot throw an error** · copy existing data into new tables or columns. The criterion that decides all of them: **nothing here is used by the current application.**	Drop old tables · drop old views · drop old columns · drop unused aliases and synonyms · drop stored procedures no longer called · apply **NOT NULL** constraints on new columns · apply **foreign key** constraints. The two constraints are the non-obvious ones, and they are non-obvious for the same reason.

WHY CONSTRAINTS CAN ONLY GO LAST

A `NOT NULL` or foreign-key constraint added *before* rollout would be enforced against instances still running the old version — which don't know they need to satisfy it. They would start throwing errors on operations that were fine a minute earlier. That breaks the principle the whole chapter rests on: **undetectability**. The users must not be able to tell a deployment is happening.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY
Migrations framework	Programmatic, version-controlled control of schema versions — roll forward, and ideally backward for testing.	Liquibase. Each change stays around as a version-controlled asset; the framework applies whatever is in the codebase but not yet in the schema.	vs **raw SQL against an admin CLI**, which has no version concept. But note the limit: a migrations framework applies changes; it does **not** make them forward- and backward-compatible. Splitting into expansion and contraction is still your job.
Shim	Code that exists only to join the old and new versions during the rollout, then is removed.	Splitting a table: an `INSERT` trigger on the old table extracts the fields and also inserts into the new one; an `UPDATE` trigger on the new table issues the matching update to the old.	From carpentry — a thin piece of wood filling the gap where two structures meet. You typically need shims for **insert, update and delete in both directions**, so about half a dozen per change. Beware the infinite loop: old insert triggers new insert triggers old insert. Delete them afterwards **as a new migration**.
Schemaless database	A store whose engine does not enforce document structure.	Documents, key-values, graph nodes.	The trap: **schemaless is a property of the engine, not of your application.** Your code expects structure. The real question is whether every old document — back to the very first customer record — is still readable by the new version. A patchwork written and re-saved by different application versions over years contains documents that have become time bombs: read one and the application raises an exception, so whatever it used to be, it effectively no longer exists.
Translation pipeline	Read any document version ever written by chaining translators until it is current.	A V2 reader detects the version and injects the document at the “V2 to V3” translator; each stage feeds the next.	Cost: every version permutation needs tests, which means keeping old documents as seed data forever, and translation time grows linearly as the pipeline deepens. If a format was ever **split**, the pipeline must split too — producing multiple documents, or writing them all back and reissuing the read (the second read needs zero translations).

Trickle, then batch	Migrate each document when it is touched, then batch-migrate the leftovers much later.	Conditional code in the new version: on load, if the document isn't current, update and save in the new format. Weeks later, batch-migrate the untouched remainder, then push a deployment removing the conditional.	vs a **single big migration**, which is fine while data is small and later takes hours you cannot afford. Trickle amortises that cost across many requests, at the price of a little latency each. Main restriction: never run **two overlapping trickle migrations against the same document type** — which may force you to split a large design change across releases. Not limited to schemaless stores; it works for any migration too long to run during a deployment.
Cache busting	Convincing browsers and intermediate proxies to fetch a new asset despite far-future expiry headers.	Static assets should carry far-future expiry — ten years is reasonable. Then version the asset: `/a5019c6f/styles/app.css`, using a git commit SHA. The specific version doesn't matter; it only has to **match between the HTML and the asset**.	Prefer the version **in the path** over a query string, so old and new versions sit in different directories and a whole version is inspectable at once. And ignore the common advice to strip the version with rewrite rules to a single unadorned file — that assumes a big-bang instantaneous switchover and breaks under a real rollout.
Session affinity	Pinning a user's requests to one server.	If the page comes from an updated instance but the asset request is balanced onto a stale one, the asset is missing. Affinity keeps a user consistently on old-app-plus-old-assets or new-plus-new.	The alternative is deploying all assets to every host *before* activating new code — which works, but means modifying running instances, so you are no longer doing immutable deployment. Easiest answer of all: serve static assets from a different cluster entirely.

The four scenarios — the reason migration order is not a matter of taste

If a rollout and a data migration run concurrently, exactly four combinations can occur:

COMBINATION	OUTCOME
An **old** instance reads an **old** document	No problem. This is just yesterday.
A **new** instance reads an **old** document	No problem — the new version was written knowing the old shape exists.
A **new** instance reads a **new** document	No problem. This is tomorrow.
An **old** instance reads a **new** document	**Uh-oh. Big problem.** The old code has never seen this shape and cannot be taught, because it is already deployed.

Only one cell fails, and its existence dictates the order: **roll out the application version first, then run the data migration.** Not the other way round, however tempting it is to “prepare the data” in advance.

> *“There'll be people who haven't logged in for a decade and have a bunch of NULLs for fields you require now. In other words, there'll be data that absolutely cannot be produced by your application as it exists today. That's why you must test on copies of real production data.”*

Test on the weird data

- Migrations fail in production because the test environment held only **nice, polite, QA-friendly data**.
- Production data has survived years of DBA actions, schema changes and application changes.
- **Do not rely on what the application currently says is legal.** Every new user picks three security questions — but records exist from before that rule, and they are still there.
- The test that matters is against a copy of real production data, precisely because it contains rows your current code could not create.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Shims (triggers bridging old and new)	Old and new instances can both read and write correctly during the rollout; nothing written late is lost.	About six per change, in both directions, plus infinite-loop risk and a removal migration afterwards. Real work — this is the price of batching changes into a release.	You must ship a structural change in one release and cannot split it.
Translation pipeline	The application can read every document version ever written.	Every permutation needs tests and old seed documents kept forever; translation time grows linearly with pipeline depth; splits force the pipeline to split.	Document versions are few, or old data genuinely must remain readable indefinitely.
One big migration during deployment	Simple; one moment, one state afterwards.	Fine while data is small; later takes minutes to hours, which is downtime you said you would not take.	Early, with small data — and knowing you will outgrow it.
Trickle, then batch	Rapid rollout with no migration downtime; the conditional check can itself be removed by a later deployment.	A little latency on every request that touches an old document; you cannot overlap two trickles on one document type.	Almost always, for any migration too big to run inside a deployment window.
Version in the asset path	Old and new versions coexist in different directories; a version is inspectable as a unit.	Build must rewrite asset URLs; you keep more than one version on disk.	Always, if the front end is coupled to back-end changes.

Session affinity for assets	Users stay coherently on one version, assets included.	Sticky sessions lengthen drain time later (part 3) and reduce balancing freedom.	Assets are served from the same cluster as the application. Otherwise, separate the cluster and avoid the problem.

> **CONCEPT BOUNDARY — EXPANSION AND CONTRACTION ARE NOT “BEFORE AND AFTER” BY CONVENTION**
>
> **It is not** a tidy convention for organising migration files.
> **It is** a compatibility rule derived from one fact: during rollout, code you cannot change is running against data you just changed. Expansion is safe because *nothing running uses the new thing yet*. Contraction is unsafe early for the mirror reason — *something running still uses the old thing*.

Anchor sketch — the only combination that breaks

Legend: boxed = a named node | **reversed = focus node** | **Q = self-test cue**

```
                    A READ HAPPENS
                          |
         +----------------+----------------+
    the INSTANCE is                   the DOCUMENT is
         |                                 |
    +----+----+                        +----+----+
   old       new                      old       new
    |          |                       |          |
    +----------+-----------------------+----------+
               |
        four combinations
               |
    +----------+---------+----------+------------+
    |          |         |          |
  old/old   new/old    new/new    old instance + NEW document
   fine      fine       fine       the one that breaks
                                            |
                                   so: ROLL OUT CODE FIRST,
                                   THEN migrate the data
```

Q Why can the failing combination not be fixed by better error handling in the old instance?
Q Which phase — expansion or contraction — would a `NOT NULL` constraint break if you got it wrong, and which principle does that violate?
Q Name the strategy that avoids the whole diagram by never having a separate migration step at all.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“Add the new columns first, delete the old ones later.”	“Split the change into an expansion phase before rollout and a contraction phase after it — nothing in expansion is used by the current application.”
“We need something to keep both tables in step for a bit.”	“Shims — insert, update and delete triggers bridging old and new in both directions, removed afterwards in their own migration. Watch for trigger loops.”
“We don't have a schema so migrations don't apply.”	“Schemaless is a property of the engine, not the application. Our code still expects structure, so old documents are a compatibility surface — some are effectively unreadable already.”
“It converts old records as it goes.”	“Trickle, then batch — amortise the migration across requests, then batch the untouched remainder, then remove the conditional in a later deployment.”
“Migrate the data first so it's ready.”	“That produces the one failing case: an old instance reading a new document. Roll the application version out first, then migrate.”
“Users are getting the old stylesheet.”	“Cache busting with the version in the path rather than a query string, so both versions coexist on disk — plus session affinity, or serve assets from a separate cluster.”
“It worked on the test database.”	“The test data was QA-friendly. We need a copy of production, because it contains records the current application could not produce.”

PART 2 · QUESTION 1

A team is splitting a customer table into two. The plan: run the migration to create and populate the new table on Friday evening, deploy the new code on Saturday morning, then drop the old columns on Saturday afternoon once the deploy is green.

(a) Name the phase each of the three pieces of work belongs to, and identify the one that is scheduled wrongly. **(b)** Name the failure that occurs between Friday and Saturday even if nothing else goes wrong, and say why writing during that window loses data. **(c)** Give the mechanism that closes that window, say roughly how many you'd need, and name the specific bug to watch for.

PART 2 · QUESTION 2

A document store holds eleven years of user profiles written by at least six generations of the application. The new release changes the profile shape again. A migration script over the whole collection takes an estimated nine hours. The team proposes running it overnight during a maintenance window.

(a) Name the strategy you'd propose instead and describe its two phases. **(b)** State its cost and its one hard restriction. **(c)** Name the alternative approach that avoids migration entirely, and give the two reasons it gets expensive over eleven years of versions.

2×2 — the compatibility matrix that dictates migration order

COLUMNS: WHICH VERSION WROTE THE DATA BEING READ? →

	Old document / old schema	**New document / new schema**
OLD instance already deployed, cannot be taught	**Yesterday. Fine.** The state the system has been in all along. *Move:* nothing. This is the baseline both other safe cells are measured against.	**The one that breaks** Old code meets a shape it has never seen. It cannot be patched — it is already running. *Move:* make this cell unreachable. Roll out the new code *before* migrating data, and use shims so writes from either side stay consistent while both are live.
NEW instance written knowing both shapes exist	**Backward compatible. Fine.** New code was written knowing old documents exist — this is the case trickle migration exploits. *Move:* on read, upgrade and save in the new format. That's the trickle.	**Tomorrow. Fine.** Both sides current. *Move:* once every instance is here and the batch has swept the remainder, remove the conditional check and contract the schema.

Read it as: three of four cells are free; the entire discipline of this part exists to make the fourth unreachable. Everything else — expansion phases, shims, trickle-then-batch, deploy-then-migrate — is a different route to the same single objective.

CARRY-OUT RULE FROM THIS PART

Only add before; only remove after. When you cannot tell which phase a change belongs to, ask the one question that decides every case: *is anything currently running using this?* If nothing is, it's expansion and it's safe now. If something still is, it's contraction and it waits.

3

PART 3 OF 4

Roll it out without being seen

Four time spans, grain size, canary batches, health checks, and putting the tools away.

TENTH-GRADER VERSION

- Update machines in small groups, never all at once, or there's nobody left to serve requests.
- Each machine needs time to stop taking new work, finish what it's doing, change, and warm up again.
- Watch the first group carefully before touching the rest — that's the one that tells you to stop.
- A machine that has started isn't necessarily ready. Sending it traffic too early looks exactly like a broken deploy.

Start with the hotel

- You're repainting a hotel that stays open. You do it floor by floor, never the whole building.
- Each floor takes four steps: **stop booking new guests** onto it, **wait for the guests already there to check out**, **paint**, and **let it air until it's actually habitable**.
- That third step is quick. The second and fourth are the ones that surprise you — a guest who booked a long stay holds the floor, and fresh paint isn't ready just because the painter left.
- And you repaint **one floor first and then look at it** before committing to the rest. That floor is your canary.

The four time spans on a single machine

SPAN	THE QUESTION	WHAT MAKES IT SHORT	WHAT MAKES IT LONG
1 · Prepare	How long to get ready to switch over?	Mutable: copying files into place so a symlink or directory reference can be flipped.	Immutable: the time to deploy a whole new image.
2 · Drain	Once you stop accepting new requests, how long until the existing ones finish?	A second or two for a stateless microservice.	A front end with sticky sessions: **session timeout plus maximum session duration** — and there may be *no upper bound*, especially if you can't distinguish bots and crawlers from humans. **Blocked threads also block the drain**, and those stuck requests look like valuable work but definitely are not. Either watch the load until enough has drained that you're comfortable killing the process, or **pick a “good enough” time limit** — the larger your scale, the more you'll want the limit, to make the whole process predictable.
3 · Apply	How long to actually apply the change?	A symlink update — very quick. For disposable infrastructure there is no “apply” at all; you bring up a new instance, so this span overlaps the drain.	Manually copying archives or editing configuration files. Slower, and more error-prone too.
4 · Ready	After starting, how long until it can take load?	A process with no warm-up.	Loading caches, warming the JIT, establishing database connections. Send load before this completes and you get server errors or very long response times for whoever is unlucky enough to arrive first.

GRAIN SIZE SETS THE CLOCK

Deployment units sit on a spectrum: **single files** (static sites, PHP, CGI scripts) → **archives** (`.rpm` , `.deb` , `.msi` , `.ear` , `.war` , `.exe` , `.jar` , `.dll` , gems) → **whole machines** (AMIs, containers, VMDKs). Copying a single file with no runtime restart will always beat copying an archive and restarting a container, which will always beat copying a gigabyte image and booting an operating system.

The consequence is arithmetic, and it is the kind of thing that ruins an evening: **it is no good planning a rolling deployment over a 30-minute window only to discover each machine needs 60 minutes to restart.**

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Drain	The span between refusing new requests and the last in-flight one finishing.	A second or two for a stateless microservice; potentially unbounded for a sticky-session front end.	vs **time-to-ready**: both are waiting, for opposite reasons. Drain is at the *end* of the old process's life; ready is at the *start* of the new one's. Confusing them is why teams budget one and get caught by the other.
Time-to-ready	The span between a process starting and being genuinely able to serve.	Loading caches, warming the JIT, establishing database connections.	vs **startup time**: the socket opens long before the work can be done. A machine that is “up” by any process check may still fail every request it receives.
Canary group	The first batch of a rollout, which you deliberately stop on.	Alpha, of Alpha through Foxtrot. Traffic is ramped to it with the load balancer while errors, latency and RAM are watched.	vs **an ordinary batch**: mechanically identical — drain, apply, health-check, re-enable. The only difference is that you *pause and look*, which is the entire value.
Health check route	An end-to-end route reporting whether an instance can actually do work.	Reports application version, runtime version, host IP, and the status of connection pools, caches and circuit breakers.	vs a **port or process check**: those confirm something is listening. A health check confirms it can serve — and doubles as the control surface that drains and re-enables the instance.
Grain size	The unit you deploy.	Files → archives → whole machine images.	vs **deployment frequency**: grain size sets the per-machine clock; frequency is what you may choose *given* that clock. Confusing the two produces the 30-minute plan with the 60-minute restart.

The rollout itself

- The constraint is fixed: enough machines must stay up to serve demand, so you cannot update them all at once — but one at a time may take unacceptably long. So you update in **batches**. Say five groups: Alpha, Bravo, Charlie, Delta, Foxtrot.
- Per group, in order: **(1)** stop accepting new requests · **(2)** wait for load to drain · **(3)** run the configuration management tool to update code and config · **(4)** wait for green health checks on every machine in the group · **(5)** start accepting requests again. Then repeat.
- **The first group is the canary.** Pause there. Use traffic shaping at the load balancer to ramp traffic up gradually while watching monitoring for anomalies — a spike in logged errors, a marked increase in latency, rising RAM. Any of those and you shut traffic off to that group and investigate *before* continuing.

- While Charlie is being updated, Alpha and Bravo are done and Delta and Foxtrot are waiting. Both versions are serving live traffic simultaneously. This is the moment all the preparation in part 2 pays for itself.

Health checks — the mechanism that makes it graceful

- Removing a machine from the load balancer pool works, but it is abrupt and may needlessly disrupt active requests.
- Better: every application and service exposes an end-to-end **health check route**. A good one reports the **application version, the runtime version, the host's IP address, and the status of connection pools, caches and circuit breakers** — useful for monitoring and debugging quite apart from deployment.
- With that in place, a simple status change tells the balancer to stop sending new work while existing requests complete.
- The same flag solves span 4: **start the service with “available” set to false**, because considerable time often elapses between a service listening on a socket and being genuinely ready to work.

Immutable rollout — the one real choice

OPTION	HOW IT GOES	WHAT IT COSTS
New instances in the existing cluster	Spin up machines on the new version alongside the old. As they come up healthy they start taking load.	You **need session stickiness**, or one caller bounces between old and new versions on different requests.
A whole new cluster, then switch	Bring up a parallel pool, check health and general well-being, then move the IP address across.	Less worry about stickiness, but **the moment of switching the IP may be traumatic to unfinished requests**.

With very frequent deployments you are better off starting new machines in the existing cluster: it avoids interrupting open connections, and it is the more palatable choice in a virtualised corporate data centre where the network is not as easy to reconfigure as in a cloud environment.

> **THE CONSTRAINT THAT HOLDS UNDER EVERY MODEL**
>
> However you roll code out, **in-memory session data on the machines will be lost** — and you must make that transparent to users. In-memory session data should only ever be a local cache of information available elsewhere. Say it as a rule: **decouple the process lifetime from the session lifetime.**

Cleanup — the job isn't done until the tools are put away

- First, **wait**. Every machine is on the new code; keep an eye on monitoring and don't swing into cleanup until you're sure the changes are good.
- **Remove the shims.** Once every instance is on the new code the triggers are unnecessary — but put the deletion into a *new migration*, not a manual drop.
- **Contract the schema** (part 2's second list), including the two constraint types that could not be applied earlier.
- For schemaless stores, this is when one-shots run: delete documents or keys no longer used, remove elements that are no longer needed.
- **Review feature toggles.** New toggles should have defaulted to off; now decide what to enable. Also look at the existing settings — any toggle you no longer need should be scheduled for removal.

> *“Why would such an omission reach production? One answer leads to the dark side. If you said, ‘Because the DBA didn't check the schema changes,’ then you've taken a step on that gloomy path.”*

Deploy like the pros — make the pipeline carry the expertise

- The example is a foreign-key constraint with no index on it. Anyone in the relational world winces; it costs hours of downtime.
- Blaming the DBA for not catching it is the wrong answer, and it leads somewhere bad. The better answer starts from “SQL is hard to parse, so our pipeline can't catch that” — because that contains the fix.
- If you accept that the pipeline should catch every mechanical error of that kind, then it follows that you should **stop specifying schema changes in SQL DDL**. Home-grown DSL or off-the-shelf migration library, it hardly matters: **turn the schema changes into data** so the pipeline has X-ray vision into them and can reject any build defining a foreign key without an index.
- The principle generalises well past schemas: **have the humans define the rules, have the machines enforce them.**

Anchor sketch — the two nested clocks

Legend: boxed = a named node **reversed = focus node** **Q = self-test cue**

```
MACROSCOPIC -- the whole deployment
+-----------------------------------------------------+
|  PREP            ROLLOUT               CLEANUP       |
|  expand schema   batches Alpha..Foxtrot  drop shims |
|  push assets     first batch = CANARY    contract   |
|  add shims           |                   toggles    |
+----------------------|----------------------------+
                       |
            MICROSCOPIC -- one machine
            +------------------------------------+
            | 1 PREP   2 DRAIN   3 APPLY   4 READY |
            |  copy    finish    flip     warm  |
            |  files   in-flight  link    caches|
            +------------------------------------+
                      ^                     ^
           the two spans that surprise you: drain has
           no upper bound with sticky sessions, and
           "started" is not the same as "ready"
```

Q Which microscopic span disappears entirely under disposable infrastructure, and what does it overlap with instead?

Q Name two distinct things that can make the drain span unbounded.

Q What single flag, set at startup, prevents the fourth span from producing user-visible errors?

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“We update a few machines at a time.”	“Batched rollout — drain, apply, wait for green health checks, re-enable, then the next group. The first group is the canary and we ramp traffic to it with the load balancer while watching for anomalies.”
“It takes a while before it works properly after starting.”	“Time-to-ready is separate from startup time — caches, JIT warm-up and connection pools. The instance reports available=false until it's genuinely ready to take load.”
“Some requests were still running.”	“The drain span. With sticky sessions that's session timeout plus maximum duration, and blocked threads will hold the drain open indefinitely.”
“We deploy whole VM images.”	“Coarse grain size — that sets the per-machine clock, and the rollout window has to be arithmetic on it, not a guess.”
“Users lost their sessions in the deploy.”	“In-memory session data is lost under every rollout model. It should only be a local cache of state available elsewhere — decouple process lifetime from session lifetime.”
“The DBA should have caught it.”	“That's the wrong fix. Express schema changes as data rather than SQL DDL so the build pipeline can reject a foreign key with no index. Humans define the rules; machines enforce them.”
“We turned the feature on for everyone at once.”	“New toggles default to off; the cleanup phase is where we review what to enable and schedule dead toggles for removal.”

PART 3 · QUESTION 1

A team plans a rolling deployment across 40 machines in a 30-minute window, in four batches of ten. In rehearsal, each machine takes about 90 seconds to apply the change. On the night, the rollout is still running three hours later, and the machines that *have* been updated are returning 500s to a fraction of users.

(a) Name the two time spans the team failed to budget for, and say why rehearsal didn't reveal either. **(b)** Explain the 500s specifically. **(c)** Give the two mechanisms needed — one for each problem — and say what a good version of the second reports.

PART 3 · QUESTION 2

A service deploys by spinning up new instances on the new version inside the existing cluster. Deploys are frequent. After a release, support reports that a small number of users saw a half-updated interface: some pages new, some old, seemingly at random, and one user reported being logged out mid-checkout.

(a) Name the missing mechanism and explain the bouncing precisely. **(b)** Name the alternative rollout topology, and state the specific cost that makes it the wrong trade here. **(c)** Explain the logout, and give the design rule that makes it a non-event.

2×2 — choosing a rollout topology

COLUMNS: DO BOTH VERSIONS SERVE LIVE TRAFFIC AT THE SAME TIME? →

	Mixed old and new both taking requests	**Switched** one cutover moment
CHANGE existing machines convergence	**Rolling batches** Alpha through Foxtrot, drain and apply per group, canary first. *Move:* budget all four time spans per machine, gate each group on green health checks, and require part 2's compatibility work — both versions are live.	**Big bang** Everything at once, in a window. *Move:* this is the thing the whole competency exists to replace. If you're here, the reason is usually that the data work in part 2 was never done.
ADD new machines immutable	**New instances, existing cluster** New machines come up on the new version and start taking load as they go healthy. *Move:* session stickiness is mandatory or a caller bounces between versions. Preferred for frequent deploys — no open connections interrupted, and kinder to a corporate network you can't easily reconfigure.	**New cluster, flip the address** Build a parallel pool, verify health, move the IP. *Move:* you get a full health check before any real traffic — but the switch moment is traumatic to unfinished requests. Better for rare, high-stakes releases than for frequent ones.

Read it as: the left column needs version compatibility; the right column needs a tolerable cutover. Frequency decides — the more often you deploy, the further left and lower you should sit.

CARRY-OUT RULE FROM THIS PART

Budget the whole clock, not just the copy. Prepare, drain, apply, ready — three of the four are usually longer than the one everyone estimates, and the rollout window is arithmetic on the slowest machine, not a guess about the fastest.

4

PART 4 OF 4

Prove it by breaking it

Chaos engineering: hypothesis, injection, targeting, blast radius — and the zombies.

TENTH-GRADER VERSION

- You can't find out how something breaks by thinking about it. You find out by breaking it, on purpose, while you're watching.
- Do it in small doses, in the real system, because a copy of the system doesn't behave like the real one.
- Things that hardly ever break are the scariest — nobody has ever learned how to fix them.
- And it applies to people too: what happens when the one person who knows how something works is away?

Start with the spare tyre

- You have a spare tyre in the boot. You've never fitted it. You've never checked its pressure. You are confident it works, because it exists.
- The confidence is **a model**: it should work. Fitting it once on a dry Sunday afternoon is **an experiment**: it does, or it doesn't, and you find out while nothing is at stake.
- That distinction is the whole discipline. Chaos engineering is **empirical rather than formal** — “We don't use models to understand what the system should do. We run experiments to learn what it does.”
- And the reason you fit it on a Sunday rather than on a motorway hard shoulder in the rain is **blast radius**.

> *Chaos engineering is “the discipline of experimenting on a distributed system in order to build confidence in the system's capability to withstand turbulent conditions in production.”*

Why it has to be the whole system, in production

- **Scale changes behaviour qualitatively.** Different ratios of instances behave differently; congested networks behave differently from uncongested ones. A system that is fine on a low-latency, low-loss network may break badly on a congested one.
- **Staging will never be full-size.** The economics forbid it — “Are you going to build a second Facebook as the staging version of Facebook? Of course not.”
- **The interesting failures are whole-system properties:** excessive retries leading to timeouts, cascading failures, dogpiles, slow responses, single points of failure.
- **Safety is not composable** — and this is the argument worth memorising, because it kills the “we tested each service” defence:
  - A client enforces a 50 ms timeout. Two providers each average 20 ms with an observed 99.9th percentile of 30 ms.
  - Called individually, either is safe with high confidence.
  - Called **in sequence**, the average still fits the 50 ms budget — but a sizable percentage of calls will not, and the client now looks unreliable.
  - Neither component changed. The unreliability is an **emergent property of the composition**, invisible in either part. Like concurrency, safety does not compose.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Chaos engineering	Experimenting on a distributed system to build confidence it can withstand turbulent production conditions.	Chaos Monkey wakes, picks an autoscaling cluster, kills one instance; the cluster should recover by itself.	vs **testing**: a test verifies a condition you already knew to check, and passes or fails. A chaos experiment carries a *hypothesis* about a steady state and can invalidate it — empirical rather than formal.
Steady state	The externally observable healthy behaviour the system maintains as a whole.	“Clustered services should be unaffected by instance failures” — a hypothesis observations invalidated almost immediately.	vs **internal metrics**: the hypothesis must be framed on externally observable behaviour, as an invariant the system upholds under turbulence — not on internals only you can see.
Blast radius	The magnitude of bad experience an experiment may cause — how many customers, and how badly.	Starting as crudely as “every 10,000th request will fail”, then moving to real victim-selection criteria.	vs an **error budget**: a budget is how much unreliability you tolerate over a period. Blast radius is how much one experiment may cause right now, and it is bounded *before* you start.
Injection	A deliberately introduced fault.	Kill an instance; add latency; fail one service-to-service call.	vs a **bug**: an injection is chosen, bounded, observed and reversible. You know when it started and can stop it — which is why a recovery plan is a prerequisite rather than a nicety.
Disaster simulation	The same discipline applied to the human system rather than the software.	A zombie apocalypse day: 50% of people designated unavailable, refusing all contact.	vs an **injection**: injections break software; simulations break roles, knowledge and availability. Both find single points of failure, but only one is usually fixed by writing documentation.

Where the idea comes from — three arguments, all worth having

IDEA	WHAT IT SAYS	WHY IT JUSTIFIES DELIBERATELY BREAKING THINGS
Drift Sidney Dekker, *Drift into Failure*	A system — people, technology and process together — lives inside three boundaries: **safety, economy, capacity**. Economic pressure, plus people not wanting to work at their productivity limit, creates a gradient pushing the whole system toward the safety boundary.	The airliner: jets fly faster higher, and faster trips mean more revenue. But at the revenue-optimal altitude, stall speed and turbulence-onset speed sit much closer together than in thicker air — less room for error exactly where the economics point. **Highly efficient systems handle disruption badly; they break all at once.** Chaos is the balancing force.
The fundamental regulator paradox Gerald Weinberg	“The task of a regulator is to eliminate variation, but this variation is the ultimate source of information about the quality of its work. Therefore, the better job a regulator does, the less information it gets about how to improve.”	Success destroys your own feedback. Paraphrased: you don't know how much you depend on your IT staff until they go on vacation. If you want information, you must reintroduce variation deliberately.
The Volkswagen microbus paradox	You learn to fix the things that break often. You never learn to fix the things that rarely break — so when they finally do, the situation is more dire.	Argues for **a continuous low level of breakage** to make sure the system can handle the big things. Related: Taleb's *Antifragile* describes systems that improve from stress — with the honest caveat that distributed information systems **don't naturally fall into that category**. Chaos is the iron a weightlifter uses: tolerable stress, deliberately applied, to build strength over time.

The Simian Army, and the choice that decides adoption

- **Chaos Monkey**: every so often it wakes, picks an autoscaling cluster, and kills one instance. The cluster should recover automatically. If it doesn't, the owning team has a problem to fix.
- It was born during Netflix's migration to AWS and microservices, when availability was being jeopardised by an increasing number of components. The choice was not robust-components *versus* robust-system — it was **and**. Stability patterns make instances likely to survive, but **no amount of code inside an instance stops AWS terminating it**.
- The key insight: AWS terminated instances often enough to be a big problem at scale, but *not* often enough that every deployment of every service got tested. So Netflix needed failures to happen **more often, until they became routine** — the same agile adage again: if something hurts, do it more often.
- Others followed: **Latency Monkey, Janitor Monkey, Conformity Monkey, Chaos Kong.** Every new kind of monkey improved overall availability. (And, per Heather Nakama at the third Chaos Community Day: people really like the word “monkey.”)

MODEL	HOW IT WORKS	WHAT IT COSTS
Opt-out Netflix	Every production service is subject to Chaos Monkey. A waiver requires sign-off, and exempt services go into a database Chaos Monkey actually consults — not a paper process. Being exempt carries a **stigma**; engineering management reviews the list and prods owners to fix their services.	Higher resistance, and people may not fully grasp what they're opting out of — or how serious it is that they didn't respond when they should have.
Opt-in	Teams volunteer their services.	**Adoption rates are much lower.** But it may be the only feasible approach for a mature, entrenched architecture with too much fragility to chaos-test everywhere.

The recommended path: **start opt-in**, create much less resistance, publicise some success stories, then move to opt-out.

> *“After a couple weeks of waiting and nagging I assumed the silence implied consent and unleashed my armies of chaos. We ended up taking QA down for a week and I pretty much ended up meeting everyone that worked at the company. Moral of the story: chaos engineering is a quick way to meet your new colleagues, but it's not a great way.”*

Prerequisites — before you kill anything

- **It can't kill your company or your customers.** Netflix had it comparatively easy: customers will press play again, and forgive almost anything except cutting off the end of *Stranger Things*. If every single request in your system is irreplaceably valuable, **chaos engineering is not the right approach for you.** You must be able to break the system without breaking the bank.
- **Limit the blast radius** — the magnitude of bad experiences, both in sheer number of customers affected and in how badly. Start as crudely as “every 10,000th request will fail”; you'll soon need more sophisticated selection and controls.
- **Trace a user and a request through the tiers**, and know whether the whole request ultimately succeeded. The trace earns its keep either way: if the request *succeeded*, it shows you where redundancy saved it; if it *failed*, it shows you where.
- **Know what healthy looks like, and from whose perspective.** Is your monitoring good enough to see failure rates go from 0.01% to 0.02% for users in Europe but not South America? Be wary that measurements may fail when things get weird — **especially if monitoring shares network infrastructure with production traffic**. As Charity Majors puts it: “If you have a wall full of green dashboards, that means your monitoring tools aren't good enough.”

- **Have a recovery plan.** The system may not return to health by itself when you switch the chaos off. Know what to restart, disconnect or clean up.

Designing the experiment

- Start with a **hypothesis**, framed as an **invariant the system should uphold even under turbulent conditions**, focused on **externally observable behaviour** rather than internals, with a healthy **steady state** the system maintains as a whole.
- Chaos Monkey's own hypothesis was “Clustered services should be unaffected by instance failures.” Observations **quickly invalidated it —** which is the point, not a failure of the exercise. Another: “The application is responsive even under high latency conditions.”
- Then check you can even *tell* whether the steady state holds right now. You may need to fix measurement first — look for blind spots like hidden delay in network switches or a lost trace between legacy applications.
- Then decide **what evidence would reject the hypothesis**. A non-zero failure rate may not be enough: if the request starts outside your organisation you already have failures from external network conditions, like aborted connections on mobile devices. You may have to dust off the statistics textbooks to decide how large a change counts as evidence.

The three injections

INJECTION	WHAT IT DOES	WHAT IT UNIQUELY FINDS
Kill an instance Chaos Monkey	Terminates an instance of an autoscaled cluster.	The most basic and crude injection. It will absolutely find weaknesses — unbounded cluster membership rosters, stale IP addresses that never leave the list, subtle configuration problems in services that had been in production for ages — but it is not the end of the story.
Add latency Latency Monkey	Slows calls down.	Two classes the killer misses: services that **time out and report errors when they should have a useful fallback**, and **undetected race conditions** that only appear when responses arrive in a different order than usual.
Fail a specific call failure injection testing (FIT)	Tags a request at the inbound edge — an API gateway — with a cookie saying “down the line, this request will fail when service G calls service H.” At that call site, G sees the cookie and reports the call failed **without even making the request**.	Vulnerability to the loss of a whole service in deep trees of service calls. Requires a common framework for outbound calls so the cookie propagates and is treated uniformly. (Not the same FIT as “framework for integrated testing”.)

TARGETING IS A SEARCH PROBLEM, AND THAT CHANGES THE STRATEGY

- **Randomness works at first** because the search space for faults is **densely populated** — most software has so many problems that shooting at random will uncover something alarming.
- Once the easy things are fixed, the space becomes **sparse but not uniform**. Some services, network segments, and combinations of state and request still hide killer bugs.
- The scale is hostile: the space is $\mathbf{2^n}$, where n is the number of service-to-service calls — so it grows doubly exponentially in the number of services. Exhaustive search is not available at any realistic size.
- Randomness also has blind spots by construction: a partner data load that runs every Tuesday corrupts data, and a service later throws a 500 rendering it. Random search is **not very likely** to find that.
- So targeting becomes human work — applying knowledge of the system, abductive reasoning and pattern matching, thinking about how a successful request works: a top-level request generates a whole tree of supporting calls; kick out one support and it may succeed or fail, and **either way you learn something**.
- Which is why you must **study the faults that did not become failures.** The system did something to stop it. Learn from the happy outcomes as much as the bad ones.
- **Cunning malevolent intelligence** (Peter Alvaro, UC Santa Cruz) automates that: collect traces of normal workload, build a database of inferences about which services a request type needs, treat it as a graph, and use graph algorithms to choose links to cut. Cut one and the request still succeeds? Then either a secondary service appeared — record it, as a human would — or the link simply wasn't crucial. A few iterations drastically narrow the search space.

Automate, and moderate

- Chaos should be **boring**. “In the best case, it's totally boring because the system just keeps running as usual.”
- When you do find a weakness, do two things: **fix that specific instance**, then **look for what else is vulnerable to the same class of problem**. A known class is what you automate against.
- With automation comes moderation — **there is such a thing as too much chaos**:
    - An instance-killer shouldn't kill the **last** instance in a cluster.
    - If you're failing calls from G to H, it isn't meaningful to **simultaneously fail every fallback G uses when H is down**.
    - One poor customer shouldn't get flagged for **all** the experiments at once.
- Hence platforms — Netflix's is the **Chaos Automation Platform (ChAP)** — deciding how much chaos, when, to whom, and which services are off-limits. The platform decides *what and when*; the *how* is usually left to an existing tool such as Ansible, which is popular partly because it needs no special agent on the targeted nodes.
- And the platform must **report its tests to monitoring**, so test events can be correlated with changes in production behaviour.

Disaster simulations — the human single points of failure

- Chaos isn't only about software faults. **Every single person in your organisation is mortal and fallible.** People get sick, break bones, have family emergencies, or simply quit without notice. Natural disasters can make a building or a whole city inaccessible.
- The question to sit with: **what happens when your single point of failure goes home every evening?**
- High-reliability organisations use drills and simulations to find systemic weaknesses on the human side exactly as on the software side. At full scale that's a business continuity exercise; it works at smaller scales too.
- The mechanism: designate some number of people “incapacitated” and see whether business continues. Made more palatable as a **zombie apocalypse simulation** — randomly select 50% of your people and tell them they're zombies for the day. They are not required to eat any brains, but they are required to stay away from work and not respond to communication attempts.
- Like Chaos Monkey, the first few runs immediately expose key processes that can't be done: a system needing a role only one person holds, or the one person who knows how to configure a virtual switch. **Record these as issues during the run**, then afterwards review them like a postmortem and close the gaps by improving documentation, changing roles, or automating a formerly manual process.
- Sequencing: **don't combine fault injection with a zombie simulation on the first run.** Once you know you can survive a normal day without people, then ramp up stress by creating an abnormal situation at 20% zombiehood.
- **The safety note that matters most:** have a way to abort. Make sure the zombies know a code word signalling “this is not part of the drill”, in case you go from learning opportunity to existential crisis.

CONCEPT BOUNDARY — CHAOS ENGINEERING VERSUS ITS THREE LOOK-ALIKES

NOT TO BE CONFUSED WITH	THE LINE BETWEEN THEM
Testing	A test verifies a condition you already knew to check, and it passes or fails. A chaos experiment is **empirical**: it has a hypothesis about a steady state and it can *invalidate* that hypothesis. Chaos Monkey's own hypothesis was invalidated almost immediately — that was the finding, not a failed test.
A test harness	A harness provokes out-of-spec failures against **one integration point** in a **test environment**. Chaos runs on the **whole system in production**, because the properties it hunts are emergent and scale-dependent and therefore invisible anywhere else.
Disaster simulation	Same discipline, different substrate: chaos injections break **software**; zombie drills and business continuity exercises break **the human system** — roles, knowledge, availability. Both find single points of failure; only one of them can be fixed by writing documentation.

Anchor sketch — the experiment loop

Legend: boxed = a named node | reversed = focus node | **Q = self-test cue**

```
PREREQUISITES  (skip these and you are not doing chaos, you are
+-- can survive it            just causing an incident)
+-- blast radius control
+-- tracing + known healthy
+-- recovery plan
      |
      v
HYPOTHESIS -- an invariant, externally observable, steady state
      |
      v
INJECT ---+--- kill instance   crude, finds plenty, not the end
          +--- add latency     finds missing fallbacks + race conditions
          +--- fail a call     FIT: cookie at the edge, G->H reports failed
      |
      v
TARGET -- random works while the space is DENSE
         then it is a search problem: a 2^n space, n = calls
         so: human reasoning, and study the faults that
         did NOT become failures
      |
      v
AUTOMATE + MODERATE -- never the last instance;
                        never all fallbacks at once;
                        never one customer for every test
```

Q Why is the reversed node the one that changes strategy over time, and what does it change from and to?
Q Which injection finds a class of bug that killing instances structurally cannot, and what are the two classes?
Q Name the prerequisite that decides whether your organisation may do this at all.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“Let's randomly kill some servers.”	“Let's run an experiment against a hypothesis about a steady state, with a bounded blast radius and a recovery plan. Killing instances is one injection, and the crudest.”
“We tested each service and they're all fine.”	“Safety isn't composable. Two providers within a 50 ms budget individually can break it in sequence — the unreliability is emergent, so it can only be observed at whole-system level.”
“We'll test it in staging.”	“Staging can't reproduce it — scale effects and congestion change behaviour qualitatively, and no one funds a full-size replica.”
“Everything's been running smoothly for a year.”	“That's the fundamental regulator paradox — the better the regulator performs, the less information it gets. And the microbus paradox says the rare failure is the one nobody knows how to fix.”
“We're running as efficiently as possible.”	“Which is drift toward the safety boundary. Highly efficient systems handle disruption badly — they tend to break all at once.”
“We'll fail that one call to see what happens.”	“Failure injection testing — tag the request at the edge, and at the G-to-H call site report failure without issuing the request.”
“Random killing isn't finding anything any more.”	“The search space has gone sparse but non-uniform. Time for targeted injection — human reasoning about the call tree, and inference from traces to narrow the space.”
“Only Priya knows how to do that.”	“That's a human single point of failure. Run a disaster simulation, record the gaps, and close them with documentation, role changes or automation.”

PART 4 · QUESTION 1

A payments platform wants to adopt chaos engineering. Every request moves money and cannot be silently dropped. The team proposes starting next week by randomly terminating production instances during business hours, having emailed all service owners that they may opt out.

(a) Name the prerequisite that should stop this plan, and say what it implies about whether they can do chaos at all. **(b)** Name the two other prerequisites missing here, and the specific risk each one covers. **(c)** Fix the adoption model, and cite the failure mode that email specifically invites.

PART 4 · QUESTION 2

A team has run instance-killing chaos for eight months. The first two months found a great deal; the last three have found nothing. Leadership concludes the system is now robust and proposes winding the programme down. Separately, last week's incident was traced to a month-end billing reconciliation writing a malformed currency field, which surfaced two days later as failed exports from a reporting service nobody had connected to billing.

(a) Give the correct interpretation of “we stopped finding things”, using the right term for what is happening. **(b)** Explain why random instance-killing was structurally incapable of finding the Tuesday bug. **(c)** Name two changes of approach, and explain what you learn from a fault that does *not* cause a failure.

2×2 — what to break, and how to choose the target

COLUMNS: HOW IS THE TARGET CHOSEN? →

	Random pick something, break it	**Reasoned** chosen from knowledge of the system
SOFTWARE	**Chaos Monkey** Pick a cluster at random, pick an instance, kill it. *Move:* start here. While the fault space is densely populated, random targeting is as good as any process and will find something alarming.	**Targeted injection** FIT on a specific service-to-service call; latency where ordering matters; cunning malevolent intelligence inferring the call graph from traces. *Move:* go here once random stops paying. The space is sparse and non-uniform, and 2^n in the number of service-to-service calls — so use human reasoning about the call tree, and mine the faults that *didn't* cause failures.
PEOPLE	**Zombie apocalypse** Randomly designate 50% of people unavailable for the day. *Move:* record every process that stalls, then run it like a postmortem. Don't combine with fault injection on the first attempt — and agree an abort code word first.	**Business continuity exercise** Remove a known single point of failure — the one person who can configure the switch, the only holder of a role. *Move:* close the gap with documentation, role changes, or automation. This is the only quadrant where the fix is usually not code.

Read it as: the left column is where you start and the right column is where a maturing programme has to go — in both rows. Teams that stay in the left column conclude they've become robust when they've actually only exhausted the easy half of the search space.

CARRY-OUT RULE FROM THIS PART

Confidence without evidence is just a model. Manufacture the failure yourself — small, bounded, observed, reversible — because the alternative is that reality picks the size, the timing and the audience for you.

SECTION 6

Concept sketch — the whole competency on one page

Three named groups, in the order the story runs: **DESIGN** → **MOVE** → **PROVE**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: DESIGN — accept the “during” | MOVE — ship into it | PROVE — break it on purpose

reversed = the one node to remember **Q = retrieval cue**

```
GROUP 1 - DESIGN . accept that "during" exists
------------------------------------------------------------------
   "planned downtime" is planned only from the inside
   the release is a SPAN, not a moment -> both versions run at once
        |
        +--> BUILD PIPELINE  a funnel; every stage looks to reject
        +--> CONVERGENCE (pets) vs IMMUTABLE (cattle)
        +--> undeployed code = inventory -> "if it hurts, do it more often"
                                 |
                                 v
GROUP 2 - MOVE . ship into a system that is already running
------------------------------------------------------------------
   DATA first, because data cannot be rolled back cheaply
        EXPAND before  ---->  roll code  ---->  CONTRACT after
        add only                batches            drop, and only
        nothing in use          canary first       now: NOT NULL, FK
             |                     |
             |            four spans per machine:
             |            prep / DRAIN / apply / READY
             |
        the one failing case: OLD instance + NEW data
        so deploy code FIRST, then migrate
                                 |
                                 v
GROUP 3 - PROVE . belief is not evidence
------------------------------------------------------------------
   safety does not compose -> must be whole-system, in production
        hypothesis -> inject -> target -> automate + moderate
        kill / latency / FIT        random while dense,
                                    reasoned once sparse
   and the same discipline on people: zombies, abort code word
```

The question per node — cover the right column

GROUP 1 — DESIGN

NODE	QUESTION	ANSWER IN ONE LINE
The “during” state	Why is designing only for “before” and “after” wrong?	It assumes the system changes in an instantaneous quantum jump. Updating takes time, and users only ever see the middle.

Build pipeline	Where does CI stop, and why is “funnel” the better word?	CI stops after a test report and an archive; the pipeline continues into trial deploys, migrations and production. Every stage exists to find a reason to reject the build.
Convergence vs immutable	What's the one-line difference, and which animal is which?	Repair from an unknown state versus rebuild from a known image. Immutable is for cattle; convergence is for pets.
The delay–risk cycle	Why do review steps make deployments riskier?	They lengthen the commit-to-production delay, which grows the batch, which is what actually carries the risk.

GROUP 2 — MOVE

NODE	QUESTION	ANSWER IN ONE LINE
Expansion	What single criterion decides whether a change belongs here?	Nothing currently running uses it. That's why adding a nullable column is safe and adding NOT NULL is not.
Contraction	Why must constraints wait until after rollout?	Old instances don't know how to satisfy them and would start failing operations that were fine a minute ago — breaking undetectability.
Shim	What is it, how many do you need, and what's the bug to watch?	Triggers bridging old and new during rollout; roughly six per change (insert, update, delete, both directions); watch for infinite trigger loops.
Reversed: old instance + new data	Why is this the node to remember?	The only one of four combinations that fails, and it can't be patched — the old code is already deployed. It dictates the order: code first, then migrate.
Trickle, then batch	What are the two phases, and the one hard restriction?	Migrate documents as they're touched, then batch the untouched remainder later. Never run two overlapping trickle migrations against the same document type.
Reversed: drain	What makes this span unbounded?	Sticky sessions (timeout plus maximum duration, unbounded if bots can't be told from humans), and blocked threads, which look like valuable work and are not.
Reversed: ready	Why isn't “started” the same as “ready”?	Caches, JIT warm-up and connection pools come after the socket opens. Start with available=false so the balancer waits.
Canary	What makes the first batch different from the other four?	You pause on it and ramp traffic gradually while watching for anomalies — errors, latency, RAM — before committing to the rest.
Health check	What should a good one report?	Application version, runtime version, host IP, and the status of connection pools, caches and circuit breakers.

GROUP 3 — PROVE

NODE	QUESTION	ANSWER IN ONE LINE
Safety is not composable	Give the example in one line.	Two providers each inside a 50 ms budget break it when called in sequence — the unreliability is emergent, invisible in either one.
Drift	Which direction does the gradient push, and why?	Toward the safety boundary — economic pressure plus people not working at their productivity limit. Highly efficient systems break all at once.

The regulator paradox	State it, and what follows.	The better a regulator does, the less information it gets about how to improve. So variation must be reintroduced deliberately.
The three injections	Name them and what each uniquely finds.	Kill (crude, finds plenty); latency (missing fallbacks, and race conditions from reordering); FIT (loss of a whole service in deep call trees).
Targeting	Why does the strategy have to change over time?	The fault space starts dense, so random works. It becomes sparse but non-uniform, and it is 2^n in the number of service-to-service calls — so reasoned targeting takes over.

Master 2×2 — the decision card you can carry to other problems

This one isn't about software, which is the point. Two questions decide whether any capability — a system, a team, a household, a hospital — is actually reliable or merely untested.

COLUMNS: HOW WELL DO YOU HANDLE IT WHEN IT DOES BREAK? →

	Badly improvised, slow, or nobody knows how	**Well** routine, rehearsed, boring
BREAKS OFTEN	**Firefighting** Constant incidents, no learning loop. Everyone is busy and nothing improves. *Move:* you don't need more breakage — you need the postmortem half. Fix the class, not the instance.	**Aim here** Failure is routine, so recovery is routine. Boring is the success condition: “in the best case it's totally boring because the system just keeps running as usual.” *Move:* keep the breakage coming deliberately, or you drift up into the top row without noticing.
BREAKS RARELY	**The dangerous quadrant** The Volkswagen microbus: you never learned to fix it, so when it goes the situation is dire. And by the regulator paradox this is where a *successful* operation lands — the better it runs, the less it learns. *Move:* manufacture failure at a survivable size. This is the entire justification for chaos engineering, fire drills, restore-from-backup tests and zombie days.	**Untested confidence** You believe you'd cope. You have no evidence — the spare tyre you have never fitted. *Move:* cheapest quadrant to fix. Run the drill once and you learn which of the two right-hand cells you were actually in.

The move that matters: you cannot control the left column by wanting to. You control the *row* — by choosing to break things yourself, at a size and time you pick. Every practice in this booklet, from canary batches to zombie days, is a way of moving down-to-up on your terms rather than reality's.

THE THUMB RULE, IN ONE SENTENCE

If it hurts, do it more often — at a size you choose, while you're watching.

Applies well outside software. A hospital runs code drills on a mannequin rather than learning on a patient. A pilot practises engine-out in a simulator rather than at altitude. A team takes a deliberate bus-factor day rather than discovering it during a resignation. Every one of them converts a rare catastrophic event into a frequent survivable one — which is the only known way to get good at it.

SECTION 7

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

Deployment symptoms

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Every release needs a maintenance window.	Designed for “before” and “after” only; the app can't tolerate a mixed fleet.	Split schema work into expansion and contraction; make the code read both shapes; then batch the rollout.
Releases are risky, so there are more reviews, so releases are rarer.	The delay–risk cycle.	Shrink the batch by increasing frequency. Run the full pipeline on every commit; move the human gate only if regulation demands it.
Deploys fail unpredictably on some machines, never reproducibly.	Convergence from an unknown state, plus state outside the recipes (kernel parameters, TCP timeouts).	Move to immutable images, or shorten machine lifetimes until effectively disposable.
Errors on operations that worked fine minutes earlier, mid-deploy.	A contraction-phase change applied too early — usually NOT NULL or a foreign key.	Move it after rollout. Old instances can't satisfy constraints they don't know about.
Records written during the deploy window have vanished.	A table split with no shims: an old instance wrote to the old table after the copy ran.	Add insert/update/delete triggers in both directions; delete them afterwards in a new migration; watch for trigger loops.
Exceptions reading old records after a release.	Old documents the new code can't parse — schemaless is an engine property, not an app one.	Translation pipeline, or trickle-then-batch. Roll code out before migrating data.
The migration would take nine hours.	A big-bang migration that has outgrown its window.	Trickle-then-batch: migrate on touch, batch the remainder later, then remove the conditional.
Users see a mix of old and new pages, or missing stylesheets.	Assets versioned by query string, or requests bouncing between versions.	Version in the path so both coexist; add session affinity, or serve assets from a separate cluster.
The rollout is running hours past its window.	Grain size and drain were never budgeted; sticky sessions or blocked threads hold the drain open.	Do the arithmetic on the slowest machine; set a drain time limit; hunt the blocked threads.
Freshly updated machines return 500s or very slow responses.	Traffic arriving before time-to-ready — caches cold, JIT unwarmed, pools unestablished.	Start with available=false and gate the load balancer on a real health check.
A bad build reached every machine.	No canary. Batches ran without a pause.	Make the first group the canary, ramp traffic with the balancer, and watch errors, latency and RAM before continuing.
A foreign key with no index reached production.	Schema changes expressed as SQL DDL, which the pipeline cannot parse.	Express schema changes as data so the pipeline can reject the build. Humans define the rules; machines enforce them.

Chaos & resilience symptoms

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
All services tested fine individually; the composed flow is unreliable.	Safety is not composable — an emergent latency-budget failure.	Test at whole-system level in production, with bounded blast radius.

Chaos testing has stopped finding anything.	The fault space has gone sparse but non-uniform — not robustness, just the easy half.	Switch to reasoned targeting; mine traces for the call graph; study faults that didn't become failures.
Everything has run smoothly for a year.	The regulator paradox and the microbus paradox together — success has removed your feedback.	Reintroduce variation deliberately: chaos injections, restore-from-backup drills, disaster simulations.
Only one person can do a critical task.	A human single point of failure.	Run a disaster simulation, record the stalls, close them with docs, role changes or automation.

SECTION 8

Lists and numbers worth remembering

ITEM	WHAT IT IS	WHY IT MATTERS IN AN ARGUMENT
Expansion list	Add table · views · **nullable** column · aliases/synonyms · stored procedures · triggers · copy data into new tables.	Every item passes one test: nothing running uses it yet. That test, not the list, is what you actually need to recall.
Contraction list	Drop tables · views · columns · unused aliases · uncalled stored procedures · apply **NOT NULL** · apply **foreign keys**.	The last two are the ones people get wrong, and both for the same reason: old instances can't satisfy a constraint they've never heard of.
The four combinations	old/old, new/old, new/new all fine; **old instance + new document** fails.	One failing cell out of four dictates the deployment order for the entire industry. Deploy code, then migrate.
The four time spans	Prepare · **drain** · apply · **ready**.	Everyone budgets “apply.” The other three are where rollouts overrun.
~6 shims per change	Insert, update and delete — in *both* directions.	The honest number that makes “let's just batch it into one release” sound expensive, because it is.
Grain size spectrum	Files (static, PHP, CGI) → archives (rpm, deb, msi, ear, war, exe, jar, dll, gem) → whole machines (AMI, container, VMDK).	Grain size sets the per-machine clock, and the rollout window is arithmetic on it — 30-minute plan, 60-minute restart is a real and common failure.
Ten years	A reasonable far-future cache expiry for static assets.	Sets up why cache busting exists at all: you deliberately made the browser stop asking.
50 ms, two providers at 20 ms	Each comfortably inside budget; in sequence a sizable percentage break it.	The compact proof that safety is not composable — it ends the “we tested every service” argument.
2^n	The size of the fault search space, where *n* is the number of service-to-service calls — doubly exponential in the number of services.	Why exhaustive chaos is not available at any realistic size, and targeting must become reasoned.
0.01% → 0.02%	The monitoring resolution question: could you see that, for Europe but not South America?	Forces an honest answer about whether you can detect an experiment's effect at all.
Every 10,000th request	A crude first blast-radius control.	Shows that starting is cheap; you don't need a platform to run experiment one.
50% zombies	The disaster-simulation dose.	Large enough to break processes, small enough to survive — and it finds the single points of failure that documentation was supposed to cover.

Glossary — the terms you should be able to define cold

TERM	DEFINITION, AND THE LINE THAT SEPARATES IT FROM ITS NEIGHBOUR
Machine / service	A machine is one configurable OS instance — metal, VM, container or unikernel. A service is a callable interface, always made of redundant copies across multiple machines. One machine is never a service.
Build pipeline	The automated path from commit to production. Distinguished from CI, which stops at the test report and archive. Best thought of as a funnel: every stage looks for a reason to reject.
Convergence	Inspect current state, plan a path to the declared end state, apply it. Starts from an unknown state, and silently ignores anything not in the recipes.
Immutable infrastructure	Never repair; rebuild from a known master image and delete the old machine. Config is injected at startup, not baked in.

Undetectability	The governing principle of a deployment: users must not be able to tell one is happening. It's what makes early constraints and early contraction wrong.
Expansion / contraction	Additive schema changes before rollout; removals and constraints after. The deciding question is whether anything running currently uses the thing.
Shim	From carpentry — thin wood filling a gap where two structures meet. Here, triggers bridging old and new during the rollout, then removed in their own migration.
Translation pipeline	Chained per-version translators bringing any historical document up to current. Costs linear translation time and permanent old seed data for tests.
Trickle, then batch	Migrate documents on touch, then batch the untouched remainder later, then remove the conditional. Not limited to schemaless stores.
Cache busting	Making browsers and proxies fetch a new asset despite far-future expiry. Version in the path, not a query string, so versions coexist.
Drain	The span between refusing new requests and the last in-flight one finishing. Unbounded under sticky sessions or blocked threads.
Time-to-ready	The span between process start and genuine readiness — caches, JIT, pools. Distinct from startup time, and the reason for available=false.
Canary group	The first rollout batch, which you pause on and ramp traffic to while watching. Distinct from an ordinary batch precisely because you stop.
Health check route	An end-to-end route reporting application version, runtime version, host IP, and the status of pools, caches and circuit breakers. Doubles as the deployment control surface.
Chaos engineering	Experimenting on a distributed system to build confidence in its ability to withstand turbulent production conditions. Empirical, not formal: experiments, not models.
Blast radius	The magnitude of bad experience an experiment can cause — both how many customers and how badly. The thing you bound before you start.
Drift	Dekker: economic pressure plus human reluctance to work at maximum productivity pushes a system toward its safety boundary over time.
Fundamental regulator paradox	Weinberg: eliminating variation removes the information you need to improve, so the better a regulator performs, the less it learns.
FIT	Failure injection testing — tag a request at the edge so a specific downstream call reports failure without being made. Not the same FIT as “framework for integrated testing.”
Opt-in / opt-out	Whether services are chaos-tested by default. Opt-out has far higher adoption and needs a real exemption database; opt-in is often the only feasible start for an entrenched architecture.

Answer key

PART 1 · Q1 — THE MONTHLY RELEASE AND THE EXTRA REVIEW

(a) The **delay–risk cycle**. Longer gaps between deployments mean more changes accumulate in each one; a bigger deployment carrying more change is riskier; when that risk materialises the natural response is to add review steps; review steps lengthen the delay. The cure feeds the disease.

(b) The proposal treats *scrutiny* as the lever when the actual carrier of risk is **batch size**. Moving from four to six weeks increases the number of changes per release by half again, and the two new gates add still more delay. It will feel safer and measurably increase the quantity of change per deployment, which is the thing that causes incidents.

(c) Go the other way: shorten the delay, shrink the batch, run the full pipeline on every commit — "if it hurts, do it more often." But that is only safe if the **application itself** can tolerate a mixed fleet: schema changes split into expansion and contraction, code able to read both document shapes, assets versioned, sessions decoupled from process lifetime. Without that, higher frequency just means more frequent outages. Credit for noting that a human pause gate is still legitimate in a regulated or safety-critical setting — but then you need an authorised approver available at 2 a.m.

PART 1 · Q2 — THE RELIABLE CONTAINERS AND THE THREE-YEAR-OLD VMS

(a) A is **immutable infrastructure**, B is **convergence**. Immutable rebuilds from a known master image and deletes the old machine; convergence inspects the current state and plans a path to the declared end state.

(b) First, convergence starts from an **unknown** state, and after three years some resources may be in a state the tool simply doesn't know how to repair — there is no path from here to the desired state. Second, and more subtle, **parts of the machine state aren't in the recipes at all** — kernel parameters, TCP timeouts and similar. The tool leaves them untouched, so they may be radically different from what you expect and different on every machine. Both mean each failure has its own history, which is exactly why they don't reproduce.

(c) B gets to run on long-lived VMs and physical hosts without image-build infrastructure. Convergence pairs naturally with **manual role mapping** ("server 2 is app"); immutable pairs with **automatic mapping** and IaaS/PaaS ("service X, Y replicas, and the platform places them"). Immutable is for cattle; convergence is for pets.

PART 2 · Q1 — THE FRIDAY MIGRATION AND THE SATURDAY DEPLOY

(a) Creating and populating the new table is **expansion** — additive, and nothing running uses the new table yet — so Friday evening is exactly right. Deploying the new code is the **rollout**. Dropping the old columns is **contraction**, and that is the one scheduled wrongly: it must wait until every instance is on the new code *and* a grace period has passed with monitoring watched. Saturday afternoon leaves no room to discover a problem, and no way back if you do.
Note what is *not* wrong here: copying data into a new table before the rollout does not violate "deploy the code, then migrate." That rule exists because an old instance may read a *document* whose shape it has never seen. In a table split, old instances go on reading the old table and never touch the new one, so the copy is safe in advance.

(b) Between Friday and Saturday, old instances are still running and still writing to the old table. Anything written after the copy completed **does not exist in the new table**. When the new code takes over and reads only the new table, those entities are simply lost — and nothing errors, which is what makes it dangerous.

(c) **Shims** — database triggers bridging the two structures. An `INSERT` trigger on the old table extracts the fields and inserts into the new one; an `UPDATE` trigger on the new table issues the matching update to the old. You need them for **insert, update and delete in both directions**, so roughly **six** per change. The bug to watch is the **infinite loop**: inserting into the old table triggers an insert into the new, which triggers an insert into the old. Remove them afterwards in a new migration, not by hand.

PART 2 · Q2 — ELEVEN YEARS OF PROFILES AND A NINE-HOUR MIGRATION

(a) Trickle, then batch. Phase one: ship the new version with a conditional on read — if the document isn't current, upgrade and save it in the new format, amortising the migration across ordinary requests. Phase two, once the most active documents are converted, batch-migrate the remainder; it's safe to run concurrently because no old instances remain. Then push a further deployment removing the conditional.

(b) Cost: a little extra latency on every request that touches a stale document. Hard restriction: **never run two overlapping trickle migrations against the same document type** — which may force you to split a larger design change across several releases.

(c) The alternative is a **translation pipeline**: chained per-version translators, with the reader detecting the version and injecting the document at the right stage. Two costs over eleven years and six generations: **every version permutation must be tested**, so old documents stay as seed data indefinitely; and **translation time grows linearly** as the pipeline deepens. Bonus credit for noting that a format split forces the pipeline to split too.

PART 3 · Q1 — THE 30-MINUTE WINDOW THAT TOOK THREE HOURS

(a) Drain and **time-to-ready**. Rehearsal measured only the *apply* span — the 90 seconds — which is the one span everybody budgets. Rehearsal didn't reveal the others because a test machine has no real in-flight sessions to drain and a cold cache nobody notices. In production, sticky sessions make drain equal to session timeout plus maximum session duration, with no upper bound at all if bots can't be distinguished from humans, and any blocked threads hold the drain open indefinitely while looking like valuable work.

(b) The 500s are the fourth span. The machines were put back into the load-balancer pool as soon as the process started, but starting a process is not the same as being ready — caches were empty, the JIT unwarmed, database connections not yet established. Whoever arrived first got errors or very long responses.

(c) Two mechanisms, not one. For the overrun: **a “good enough” drain time limit** rather than waiting for the load to fall to zero — at scale you want the limit precisely to make the process predictable, and no health check will bound a drain that sticky sessions have made unbounded. For the 500s: a proper **health check route**, with the instance starting at **available=false** and flipping to true only when genuinely ready; the same flag makes the drain graceful, since a status change tells the balancer to stop sending new work while existing requests complete, far gentler than yanking the machine from the pool. A good health check reports the **application version, the runtime version, the host IP address, and the status of connection pools, caches and circuit breakers**.

PART 3 · Q2 — THE HALF-UPDATED INTERFACE

(a) Session stickiness is missing. New instances came up in the existing cluster and began taking load as they went healthy, so successive requests from one user were balanced across both versions — page from a new instance, asset or subsequent page from an old one, apparently at random. This is the known cost of spinning new instances into an existing cluster.

(b) The alternative is starting a **whole new cluster and switching the IP address** once the new pool is verified healthy. Stickiness matters much less there. The cost that makes it wrong here is that **the moment of switching the IP is traumatic to unfinished requests**, and with *frequent* deploys you'd pay that repeatedly. Starting new machines in the existing cluster avoids interrupting open connections and is also the more palatable option in a corporate data centre where the network isn't easily reconfigured.

(c) The logout is in-memory session data lost when an old instance went away — and that happens under **every** rollout model, so it can't be designed around by choosing a different topology. The rule: **in-memory session data should only ever be a local cache of information available elsewhere; decouple the process lifetime from the session lifetime.** Then losing an instance costs a cache miss, not a checkout.

PART 4 · Q1 — CHAOS ON A PAYMENTS PLATFORM

(a) The first prerequisite: **your chaos engineering can't kill your company or your customers.** If every single request is irreplaceably valuable, chaos engineering in this form is **not the right approach**. Netflix could rely on customers pressing play again; a payment that vanishes is not comparable. The honest conclusion is either “not this, not yet” or a drastically narrowed scope — non-critical paths, or a lower environment — not a softer version of the same plan.

(b) Missing: **blast-radius control** (there's no victim-selection criterion at all — even something as crude as “every 10,000th request” would be a start), and **tracing plus a definition of healthy** (you need to follow a user and a request through the tiers and know whether it ultimately succeeded, and to know whether monitoring could even detect the effect — 0.01% to 0.02% for one region). Also credit **a recovery plan**: the system may not return to health when the chaos stops. And note that monitoring may itself fail when things get weird, especially if it shares network infrastructure with production traffic.

(c) Start **opt-in**, not opt-out. Opt-in has much lower adoption, but it creates far less resistance and lets you publicise success stories before moving to opt-out — and for a mature, entrenched architecture it may be the only feasible starting point. The failure mode the email invites is exactly Nora Jones's story: silence was read as consent, chaos was unleashed, QA went down for a week. If you start with opt-out, people may not understand what they're opting out of, or how serious it is that they didn't respond.

PART 4 · Q2 — EIGHT MONTHS OF MONKEYS, AND THE MONTH-END BUG

(a) Finding nothing is **not** evidence of robustness. The fault space was **densely populated** at the start, which is why random targeting paid so well; it has become **sparse but not uniform**. Some services, network segments, and combinations of state and request still hide latent killer bugs. Winding the programme down at exactly this point mistakes exhaustion of the easy half of the search space for having searched it.

(b) Because it is a **search problem with hostile dimensionality** and the bug sits in a thin, time-dependent slice of it: a job that runs once a month, corrupting data whose effect surfaces two days later in an apparently unrelated service. Random instance-killing samples the wrong dimension entirely — it perturbs instance liveness, not data validity, and has no way to reach a fault that only exists once a month. The space is 2^n in the number of service-to-service calls, so exhaustive random search was never going to reach this one.

(c) First, move from random to **reasoned targeting**: apply knowledge of the system, think about the tree of calls supporting a top-level request, and use other injections — latency (which finds missing fallbacks and order-dependent race conditions) and **FIT** (which fails a specific service-to-service call in a deep tree). Second, mine traces to build the call graph and choose links to cut — Peter Alvaro's “cunning malevolent intelligence” — which narrows the space in a few iterations. On faults that don't cause failures: **the system did something to prevent it**, and the trace shows you where the redundancy lives. That's a positive finding about your real architecture, and studying it is how you learn where the crucial calls actually are.

HOW TO SCORE YOURSELF

Two marks, both quick. **Completeness:** did you name every element, and get the *ordering* right — expansion before, contraction after, code before data, canary before the rest? Ordering is the thing this competency is actually about; missing it matters more than missing a term. **Phrasing:** could you have said your answer in a design review without hand-waving? If you wrote “we'd lose data” where the key says “an old instance writes to the old table after the copy, so the entity never reaches the new table,” the idea is there but the mechanism isn't — and the mechanism is what persuades people.

SECTION 10

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 · The “during” state — why before/after design fails						
2 · Build pipeline vs CI, and why “funnel”						
3 · Convergence vs immutable — and the two failure modes						
4 · Manual vs automatic role mapping						
5 · The delay–risk cycle and the way out						
6 · Expansion list, and the criterion behind it						
7 · Contraction list, and why constraints wait						
8 · The four old/new combinations, and the order they dictate						
9 · Shims — what, how many, and the loop bug						
10 · Translation pipeline vs trickle-then-batch						
11 · Cache busting, asset versioning, session affinity						
12 · The four time spans, and which two surprise you						
13 · Grain size and rollout arithmetic						
14 · Batches, canary, health check contents						
15 · Existing cluster vs new cluster rollout						
16 · Cleanup: shims, contraction, feature toggles						
17 · Safety is not composable — the 50 ms example						
18 · Drift, regulator paradox, microbus paradox						
19 · Chaos prerequisites and blast radius						
20 · The three injections and what each finds						
21 · Targeting as a search problem						

22 · Disaster simulations and the abort word						
23 · Master matrix — breakage frequency × recovery skill						

Final quiz — no notes, fifteen minutes, write the answers

1. State the one criterion that decides whether a schema change belongs in the expansion or contraction phase, and apply it to: adding a nullable column, adding a foreign key constraint, and copying data into a new table.
2. Of the four combinations of old/new instance against old/new data, name the one that fails, say why it cannot be fixed in the old instance, and give the deployment ordering rule it dictates.
3. Name the four microscopic time spans for updating one machine, and say which one disappears under disposable infrastructure and what it overlaps with instead.
4. Give two distinct reasons a drain can take unbounded time, and one reason a freshly started instance returns errors.
5. Explain in two sentences why adding review steps to a risky deployment process makes deployments riskier.
6. Your colleague says “we're schemaless, so we don't need migrations.” Give the two-sentence reply.
7. Describe trickle-then-batch in three steps, state its cost, and state its one hard restriction.
8. Two services each respond in 20 ms average, 30 ms at the 99.9th percentile, against a 50 ms client timeout. Explain what goes wrong and what general principle it demonstrates.
9. Name the three chaos injections and one class of bug that latency injection finds which instance-killing structurally cannot.
10. A team's chaos programme has stopped finding bugs. Give the correct interpretation and the two changes of approach that follow.